ME 469: Machine Learning and Artificial Intelligence for Robotics

Assignment 1: Search and Navigation

By: Tomasz Krzeminski
Date: 10/26/2025

1 Part A

1.1 Low Resolution Grid (*Step 1*)

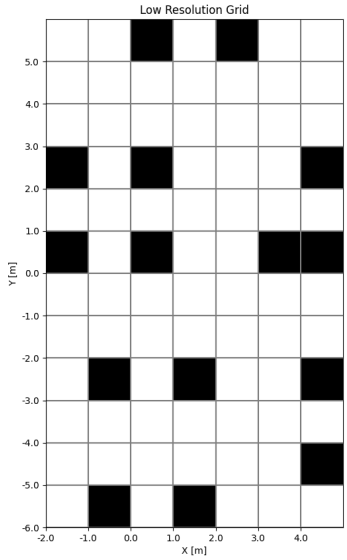

Figure 1: *1x1m* Resolution Grid | *x:[-2,5], y:[-6,6]*

In order to perform search and navigation, the simulation will first require a discrete space representation (i.e. a map). To simplify the problem, a low resolution grid, as seen in Figure 1, was programmed and visualized in python. The grid itself is a 2D array with a range of x:[-2,5], y:[-6,6], and a resolution of 1m per grid cell. For ease of computation and representation, each cell is populated with a 0 if it is unoccupied, and a 1 if there is an obstacle present. The grid was populated with obstacles, whose locations were provided from a landmark dataset. The connectivity of this world is 8-neighbor based, meaning that the navigator is only able to observe and move to one of the 8 cells surrounding its position. This notion also means that the world is partially observable, however, this is omitted for the time being.

1.2 A* Algorithm, Heuristic and Potential Fields (*Step 2*)

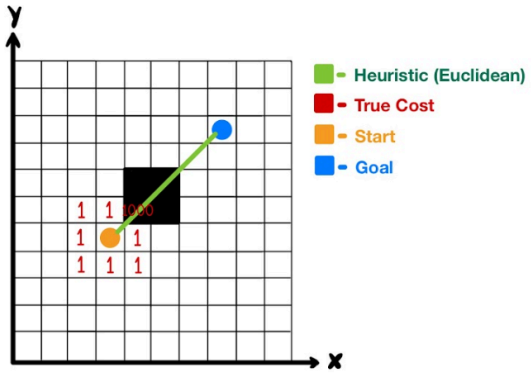

Figure 2: A* Algorithm with True Cost and Heuristic Visualized

The A* search algorithm is a complete and optimal search algorithm that employs an evaluation function, as seen in Equation 1, which is a representation of the cost of the best path from the robot's current position (n) to the goal (Russell & Norvig, 2020, p.103).

$$f(n) = g(n) + h(n) \tag{1}$$

The true cost function, *g(n)*, is the cost it takes for a robot to move from its current state to the next state. In the case of this world, the true cost for moving into an empty cell is 1, and the cost of moving into a cell occupied by an obstacle is 1000, as seen in Figure 2. This true cost allows the robot to treat movement to any cell equally, but dissuades it from moving into a cell with an obstacle, which would result in a suboptimal traversal given its cost.

$$g(n) = \begin{cases} 1 & if\ n\ Unoccupied \\ 1000 & Otherwise \end{cases} \tag{2}$$

In order for A* to be complete and optimal, an admissible heuristic function, *h(n)*, must be used. A heuristic function is said to be admissible if it does not overestimate the total cost to the goal at any point in the traversal space (Russell & Norvig, 2020, p.104). As such, I decided to utilize the Euclidean distance measurement, which is seen in Equation 3.

$$h(n) = \sqrt{(x_{Goal} - x_n)^2 + (y_{Goal} - y_n)^2} \tag{3}$$

To check whether this choice of a heuristic function really is admissible, we can take the scenario from Figure 2. In this situation, the true cost to traverse from the starting position to the goal position is 7, which can be determined by counting the smallest number of cells needed to reach the goal. To compute the Euclidean distance between the two points, we can assume that the start position is (0,0), and the goal position is (4,4) as determined by propagating this coordinate system in Figure 2.

$$h((0,0)) = \sqrt{(4-0)^2 + (4-0)^2} = \sqrt{32} = 5.66 \tag{4}$$

Considering that 5.66 is less than 7, it can be concluded that the heuristic function is admissible for this scenario. As a matter of fact, this continues to be true if Equation 5 is met for every cell in our world, and therefore it would make this heuristic function fully admissible for this problem.

$$h(n) = \sqrt{(x_{Goal} - x_n)^2 + (y_{Goal} - y_n)^2} \tag{5}$$

Another navigational method in robotics is Potential Fields, which employs a field of attractive and repulsive forces to guide a robot from a start position to a goal position, while avoiding any obstacles present in the environment (Khatib, 1986, p. 91). This approach creates a potential field of the environment by parsing through each grid cell, and computing the attractive contribution of the goal position, as well as all the obstacles in the environment. An example of such a potential field can be seen in Figure 4. The attractive potential field contribution can be seen in Equation 6 and the repulsive potential field contribution from each obstacle is seen in Equation 7. The total potential field at a specific coordinate is calculated via Equation 8.

$$U(X)_A = C\sqrt{(x_{Goal} - x)^2 + (y_{Goal} - y)^2} \tag{6}$$

$$U(X)_R = \sum_{i=0}^{N} \frac{1}{\sqrt{(x_{Obs,i} - x)^2 + (y_{Obs,i} - y)^2}} \tag{7}$$

$$U(X)_{Total} = U(X)_A + U(X)_R \quad (8)$$

By computing the total potential field, the robot is able to navigate the gradient formed by the field, and eventually find the goal position. Despite the relative simplicity of this method, however, there exist a number of challenges. Firstly, gradient descent methods, such as this, can experience local minima at which the robot can get stuck at. Secondly, this implementation of potential fields requires a precomputed potential field of the environment, which means it cannot be used in an online application.

1.2.1 A* Path Planning and Potential Fields (*Step 3*)

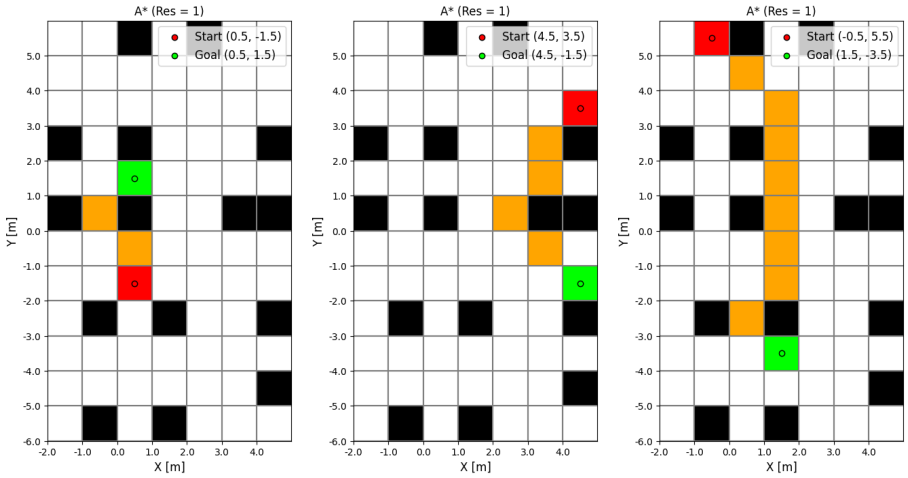

Figure 3: Paths Generated by A* for Low Resolution Grid

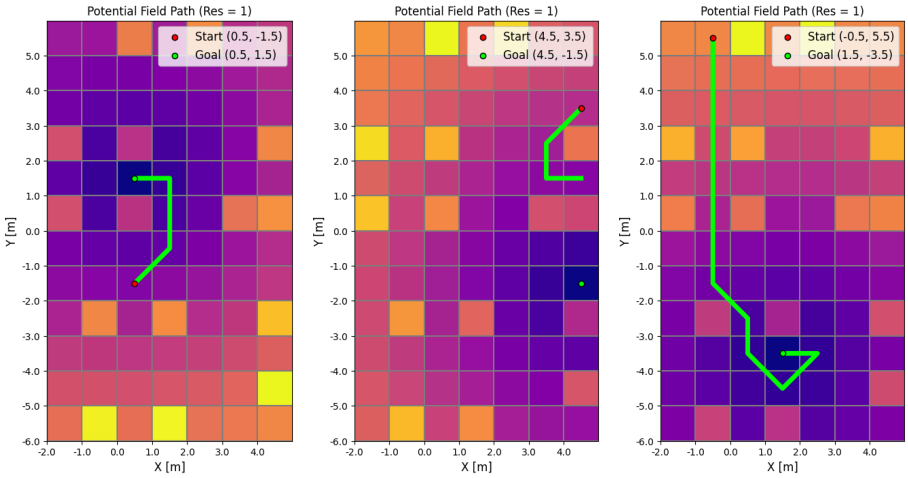

Figure 4: Paths Generated by Potential Fields for Low Resolution Grid (C = 0.5)

The A* paths in Figure 3 show that the algorithm is optimal and complete by the fact that it finds the shortest path to each goal. This also means that the heuristic is admissible as each path is optimal, and takes the least amount of steps to reach a goal position. The potential field paths in Figure 4 show that most paths are able to find the goal position, however, the middle graph shows that the robot gets stuck in a local minima, and is unable to get out. Furthermore, it can be seen from these plots that the paths are not optimal as the robot takes unnecessary steps like those seen in the right plot.

1.3 Online A* Algorithm (*Step 4*)

The traditional A* implementation is known as an offline search algorithm due to the fact that the navigator/robot has an *a priori* knowledge of all the obstacles in the environment. More specifically, the robot has a complete representation of the world, and as such, is able to plan an optimal path. This however, is not representative of true path planning and navigation as robots will not always have complete knowledge of their environment. This is the case in our problem when partial observability is applied. For this reason, there exist modified A* implementations capable of running online, and therefore allowing the robot to navigate and learn as it moves. Because of the partial observability assumed for online A* variants, the set of expanded nodes is limited to the immediate 8 neighbor cells, which means the robot will have to check a path before knowing if it is optimal.

The online A* variant that I decided to use for this assignment, is known as the Learning Real-Time A* (LRTA) algorithm. This implementation dynamically updates an internal representation of the world as the robot moves and computes costs of cells it traversed to. The similarities between LRTA* and A* are that both know the starting position, the goal position, the true cost of the world, and an admissible heuristic for distance. Besides this, LRTA* utilizes a Results table and a Cost table (H). The results table maps pairs of actions and previous states to resulting states, ultimately acting as the algorithm's memory of visited locations. The cost table maps a state to a cost, which is computed by the A* evaluation function (1), if the state is known, or simply the heuristic distance if the state has not yet been determined. These tables are continuously updated as the robot traverses the world, which means that the robot has to visit the state it believes is optimal before knowing if its path is blocked by an obstacle. This means that the paths generated by LRTA* will be complete, however, they will not be optimal.

1.3.1 Online Path Planning (*Step 5*)

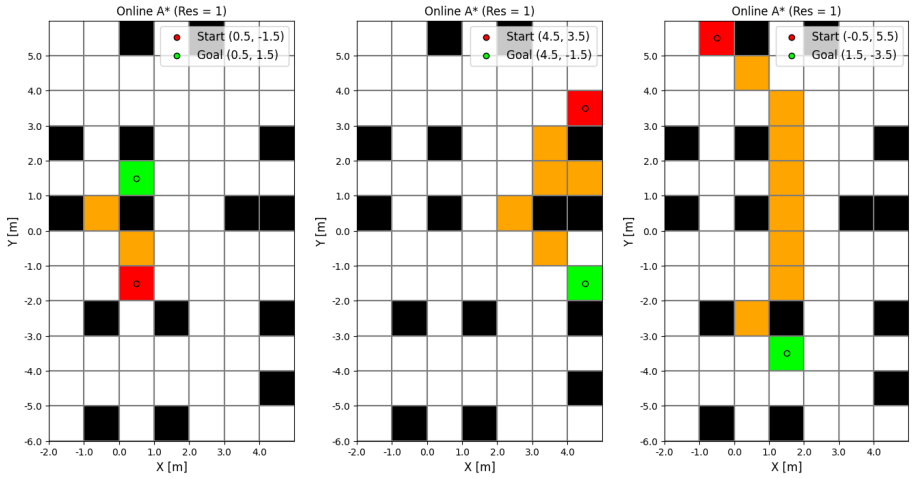

Figure 5: Paths Generated by Online A* for Low Resolution Grid

Figure 5 shows that the online A* implementation creates complete paths, as each path was able to find the goal position. Furthermore, the leftmost and rightmost plots show that the algorithm is capable of finding optimal solutions when the environment is not very complicated or cluttered. This, however, is not the case for the middle plot, as it can be seen that the algorithm first expanded a node sandwiched between two obstacles, as it believed that a direct path exists to the goal. Once it visited the node, however, it saw that an obstacle was present, and had to take a different path. As such, the online A* is complete but not optimal, which is an expected result given partial observability.

1.4 High Resolution Grid (*Step 6*)

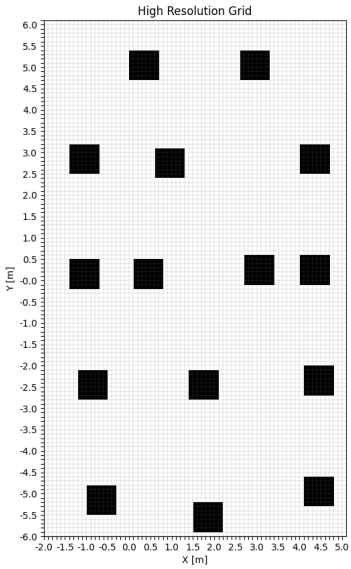

Figure 6: 0.*1x0.1m* Resolution Grid | *x:[-2,5], y:[-6,6]*

To further explore the path planning capabilities of the online A* algorithm, a higher resolution grid representation of the world was implemented. This increased resolution required that the obstacles be inflated to account for the physical size of the robot. In order to achieve this inflation, the centers of the obstacles were extracted from the landmark dataset and plotted in the finer resolution grid. An inflation kernel was then applied to the obstacle centers, which resulted in the obstacles taking on a 0.7x0.7m area. This higher resolution grid offers a more complex environment due to the fact that there exist a larger number of cells to explore.

1.5 High Resolution Online A* and Potential Fields Path Planning (*Step 7*)

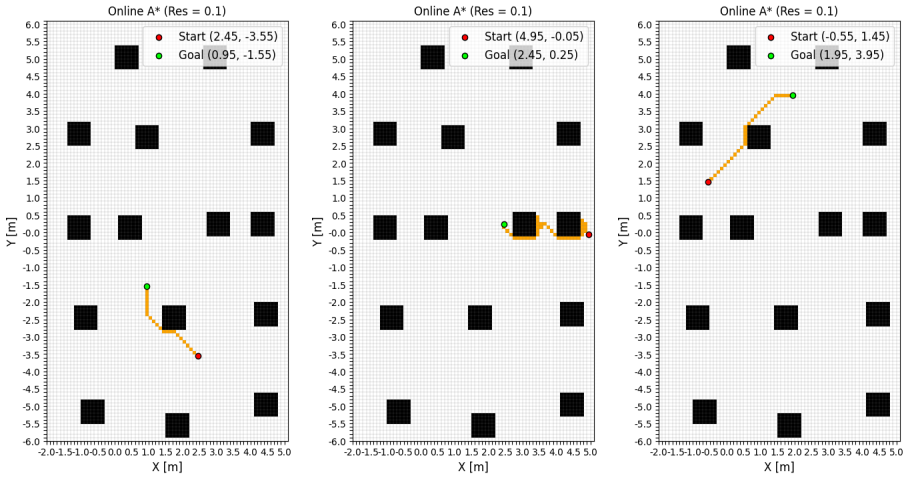

Figure 7: Paths Generated by Online A* for High Resolution Grid

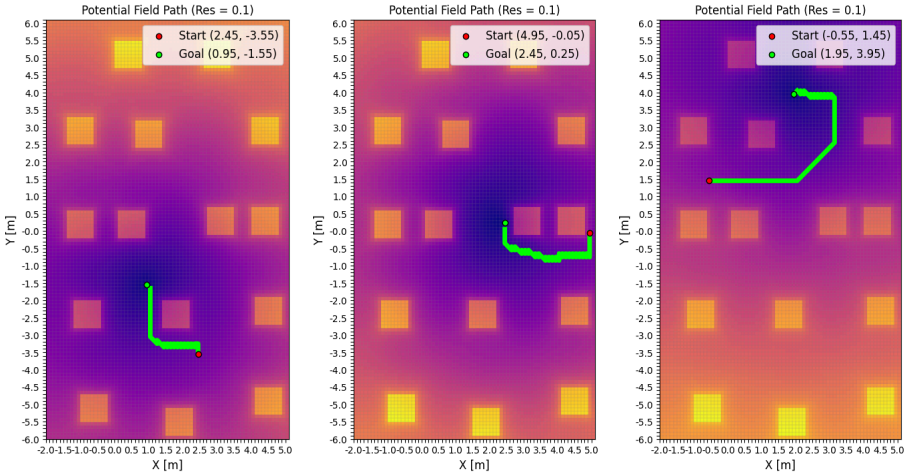

Figure 8: Paths Generated by Potential Fields for High Resolution Grid (C = 0.5)

From Figure 7, it can be seen that the online A* algorithm is able to find a path from the start to goal position in each plot. Similar to what was seen in 1.3.1, the leftmost and rightmost plots show that the algorithm is able to find optimal paths, however, the middle plot again shows that the algorithm gets stuck behind obstacles, and has to check more tiles before finding the optimal path. This is expected as the algorithm has no *a priori* knowledge of the obstacles, and how much space they span. Figure 8 showcases the potential field paths, and again, it can be seen that the paths are not optimal. In this case, however, the paths are complete, as the gradient traversal experienced no local minima. This improved performance is likely due to the higher resolution of the world, which more finely calculates the potentials at each cell.

2 Part B

2.1. Offline Navigation Controller (*Step 8*)

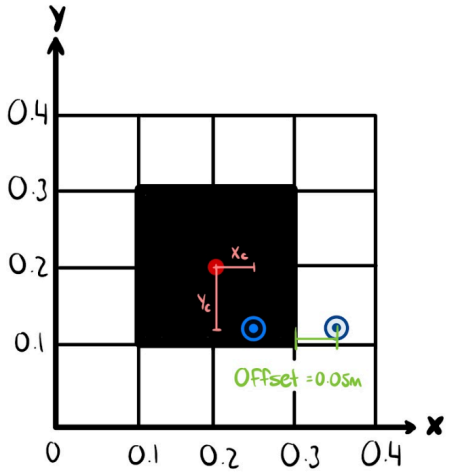

Figure 9: Collision Avoidance Process

To implement navigation for a mobile robot, the paths generated by the online A* algorithm in 1.5 (Figure 7), can be preprocessed by first applying an *nth* order Bezier curve (to the path, which is specified by Equation 9. Collision avoidance is then done by shifting the trajectory point in the direction closest to the obstacle center by an offset specified by the user. An example of this collision avoidance procedure is shown in Figure 9, where the trajectory point is closer to the center in the x-direction as demonstrated by x_c , and

therefore, offset away from the obstacle in that direction by 0.05m. The curve generated by this obstacle avoidance is noisy and rough, which requires that it is again smoothed by another Bezier curve.

$$B(t) = \sum_{i=0}^{||t||} \sum_{j=0}^{n} P_i \binom{n}{j} (t_i)^j (1 - t_i)^{n-j} \tag{9}$$

The following equation specifies an *nth* order Bezier curve, whose input is a path, *t,* which contains the control points of the Bezier curve. P_i specifies the control points used for defining the curvature of the curve. The combination specified by n Choose j specifies all the possible combinations of control points for the curve. The remaining polynomial terms are used to specify where the control points lie in space (Vail, 2016). The complete Bezier curve outputs a smoothed path, whose trajectory points were determined by iterating through Equation 9.

With the preprocessed trajectories generated, the robot will use a tuned PID controller to generate control velocities, and then traverse the world by means of a motion model implemented in the previous assignment. The error terms needed for determining the linear and angular velocities can be derived from the components specified in Figure 10. Firstly the linear error is determined by taking the distance between the center of the robot and the control point on the trajectory, as specified by Equation 10.

$$e_v = \sqrt{(x_d - x_t)^2 + (y_d - y_t)^2} \tag{10}$$

The angular error is determined by taking the difference between the heading of the robot, specified by θ_t and the desired heading, specified by θ_d as seen in Equation 12. The desired heading is computed by taking the arctangent of the y-distance divided by the x-distance, as specified by Equation 11.

$$\theta_d = tan^{-1}\left(\frac{y_d - y_t}{x_d - x_t}\right) \tag{11}$$

$$e_w = \theta_d - \theta_t \tag{12}$$

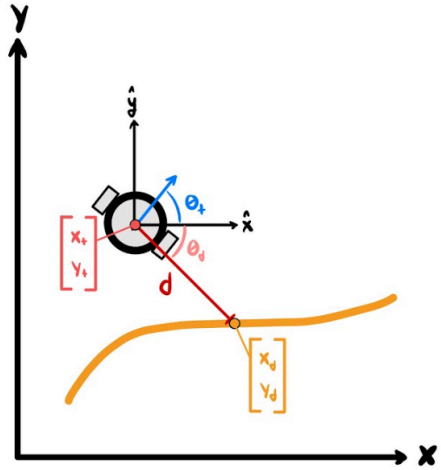

Figure 10: Trajectory Following Error Term Components

With the error terms computed, the derivative of the error terms and the integral of the error terms can be determined by taking the derivative and integral respectively. Once all of these terms are known, the commanded linear and angular velocities can be determined through Equations 13 and 14. This control loop is performed for each point on the smoothed trajectory, which allows the robot to navigate its environment based on a preprocessed trajectory. To further enhance the realism of simulation, acceleration limits were enforced

based on the real acceleration of a physical mobile robot (a = 0.288$\frac{m}{s^2}$, α = 5.579 $\frac{rad}{s^2}$), and a sampling rate of dt = 0.1s was used.

$$v = K_{pv} e_v + K_{iv} \int e_v + K_{dv} \dot{e}_v \tag{13}$$

$$w = K_{pw} e_w + K_{iw} \int e_w + K_{dw} \dot{e}_w \tag{14}$$

2.1.1 Offline Navigation Performance Analysis (*Step 9*)

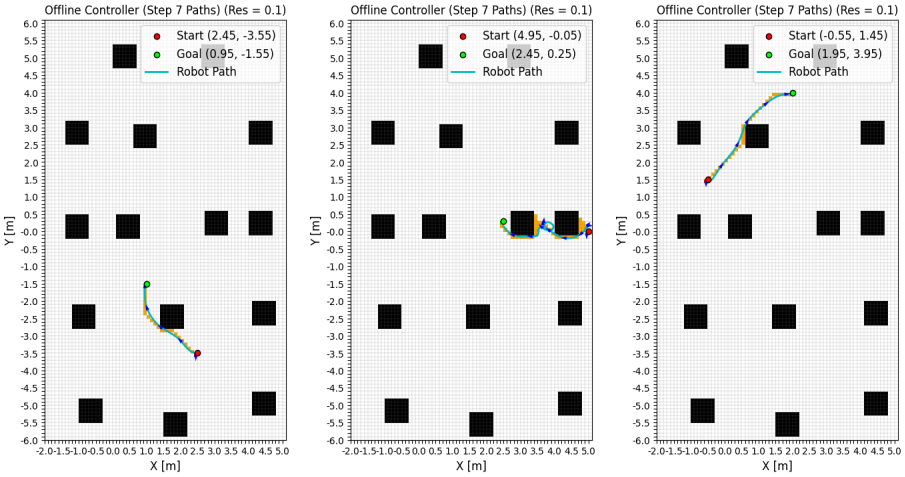

Figure 11: Robot Following Preprocessed Trajectory based on Paths from Step 7

To analyze the performance of the offline PID controller, the paths generated in 1.5 (Figure 7) were used as the input trajectories. The robot's initial position was set to be the start position, and its heading was initialized to $\theta_t = \frac{-\pi}{2}$ rad. The starting velocities were also initialized to zero, meaning that the robot was started from stationary. This simulation also assumes that the robot knows its exact state once it performs a motion, however, it does not know where it will end up while taking an action. This means that the robot does not have any uncertainty about its position, however, moving is noisy and uncertain. For this reason, the motion model from the previous assignment utilized a noise parameter set to α = 0.5 for all simulations.

From the plots in Figure 11, it can be seen that the robot, modeled by the cyan line, is capable of colliding with obstacles and is able to find the goal position. Furthermore, it can be seen that the PID controller is tuned very well and the robot is able to converge to input trajectory immediately. This is evident by the fact that the heading is facing downward at the start, and is very quickly able to turn to drive the input trajectory. A major challenge that occurred during the implementation of this controller was preprocessing the trajectories to avoid collisions, while following tangible curvature. For example, the middle plot in Figure 11 showcases the path of the robot, in which it can be seen that the robot performs a tight turn in the middle of the path. This occurred because the Bezier curves produced a path that caused the robot to have to turn too quickly. Finally, while collisions were generally avoided, there still existed minor clipping into the obstacles due to the fact that the robot did not know where it would end up after performing an action. Overall, the offline PID controller worked very well once the trajectory preprocessing was tuned and the PID controller was tuned. It is also good to note that no instability occurred once everything was tuned.

2.2 Online Navigation Controller (*Step 10*)

Finally, to tie everything together, a new online PID controller was developed to allow the robot to perform search and navigation at the same time. The major assumptions from 2.1.1 were maintained in this part of the simulation. To bring the controller online, no preprocessed trajectories were fed to controller and instead, the robot ran the online A* algorithm discussed in part 1.3. The general control loop of the online controller is as follows. First, the online A* algorithm generates a position to visit. This position is then used as the desired point for the PID controller to reach, and as such the linear and angular errors are determined based on this position. The PID control then loops until the position of the robot is within a threshold of the commanded position. Lastly, the control loop runs until the robot's position is within a specified threshold of the goal position. To further enhance the performance, collision avoidance was implemented similar to that discussed in part 2.1. The only modification done to the collision avoidance was that instead of offsetting the robot's position when collision was detected, the robot would simply be unable to move into an obstacle depending on which face it is contacting. This would result in the robot simply sliding along the edge of the obstacle.

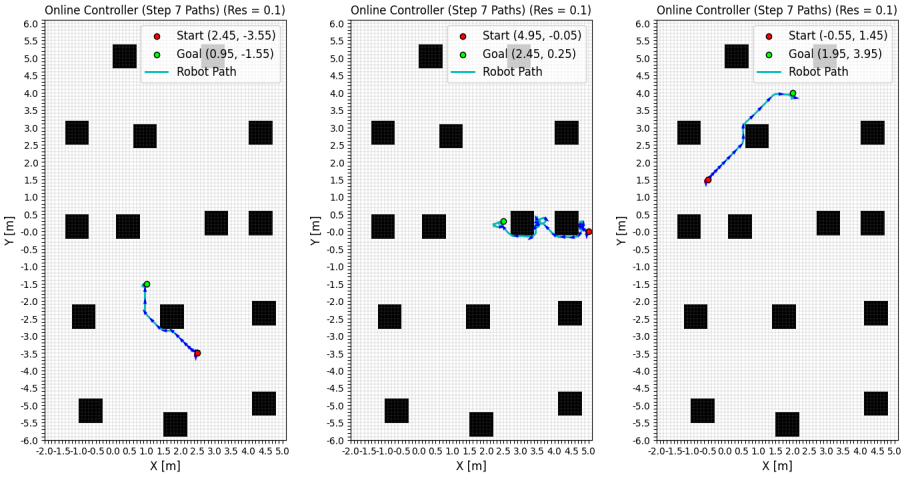

Figure 12: Robot Performing Search and Navigation Online

Figure 12 shows the online PID controller working, allowing the robot to search and navigate its environment. From the leftmost and rightmost plots, it can be seen that the robot is able to very easily navigate towards the goal position, and is able to avoid clipping into the obstacles by sliding along the obstacle, thanks to the collision avoidance discussed above. The middle plot demonstrates the capability of this controller as the robot has to navigate between two obstacles which have a small separation between them. Similar to the results seen in Figure 7, the robot explores along the edge of the obstacle, until it understands its environment better, and is forced to turn around to find the more optimal path. In this middle plot, it can be seen that the robot physically turns around once it has explored the obstacle face, and begins to follow the optimal path. Similar to the previous plots, it can be seen that the robot avoids obstacles, however, minor clipping occurs at the corners of the obstacles. This is likely due to the movement noise discussed in part 2.1.1. Another thing to address about this performance is that the robot sometimes has to perform multiple movement loops before it converges to a point. This is explicitly visible in the middle plot, where the robot circles around the goal position until its position is within a threshold distance to the goal. Nonetheless, this performance is very good considering that there is noise in the system, and the robot is planning, learning and navigating on the go.

2.2.1 Online Navigation Performance Analysis (*Step 11*)

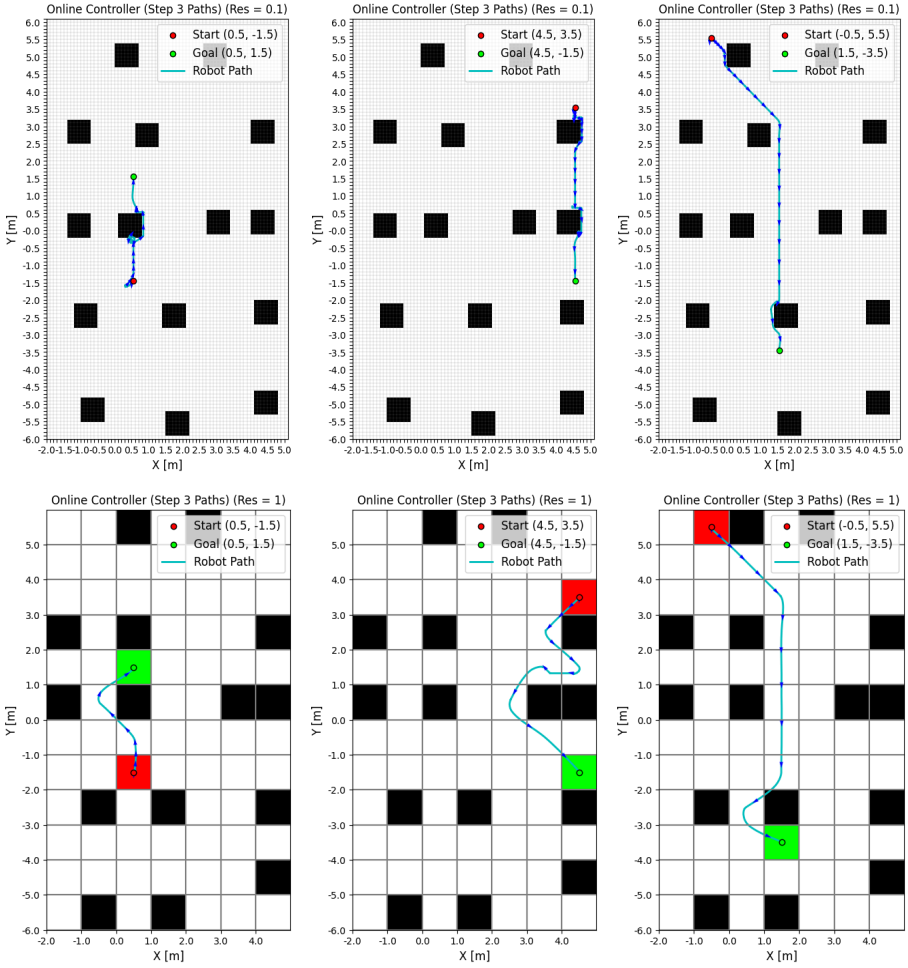

Figure 13: Robot Performing Search and Navigation Online in High and Low Resolution Grids

Similar to what was seen in part 2.2, the high resolution grid performance is consistent, and the robot is able to find the goal position each time. The top left plot in Figure 13 does show a little more clipping into the obstacle, but again, this can be associated with the motion noise. When observing the low resolution grid performance, it can be seen that the robot moves with a lot more certainty, as the commanded velocities are a lot larger. This is indicated by the fact that the spacing between the display robot (represented by blue arrows) is a lot larger. The performance of the low resolution grid is also a lot better, as the robot only has to check only 1 cell to determine if an obstacle is present or not. This is not the case for the high resolution as the robot has to traverse along the face of an obstacle before it can determine if the path is optimal. This inherent difference is emphasized in the bottom left plot as the robot very quickly turns to avoid the obstacle, and finds the goal position. In contrast, the top left plot shows the robot getting stuck around the obstacle, causing it to traverse along the face. The advantage of a coarser grid is that the robot needs to visit less cells, but in turn it misses finer details, leading to less optimal paths. A finer grid, on the other hand, allows for more optimal paths, but has to check more cells, which increases the number of cells to visit, and sometimes more navigation/exploration.

12. Other Consideration (*Step 12*)

The controllers worked very well, however, in the real world the robot would experience motion noise, and would also not know exactly where it is. This would result in faulty navigation unless a filter was applied to improve state estimation. Furthermore, the simulation causes the robot to collide with obstacles sometimes, which in the real world would result in the state of the robot to change. Finally, we assume perfect control velocities, which is not guaranteed in the real world due to motor inefficiency and battery charge.

References Cited

Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A modern approach, 4th us ed.* Artificial Intelligence: A Modern Approach, 4th US ed. https://aima.cs.berkeley.edu/

Khatib, O. (1986, Spring). Real-time obstacle avoidance for manipulators and Mobile Robots | IEEE conference publication | IEEE xplore. https://ieeexplore.ieee.org/document/1087247/

Vail, N. (2016, May 24). *Nerding out with bezier curves*. freeCodeCamp.org. https://www.freecodecamp.org/news/nerding-out-with-bezier-curves-6e3c0bc48e2f